



UNIVERSITÀ  
degli STUDI  
di CATANIA

DIPARTIMENTO DI INGEGNERIA  
ELETTRICA ELETTRONICA E INFORMATICA

CORSO DI LAUREA IN INGEGNERIA INFORMATICA

---

*Giuseppe Ravesi*

---

*SVILUPPO DI UNA PIATTAFORMA AMMINISTRATIVA PER LA GESTIONE  
EFFICIENTE DI E-COMMERCE UTILIZZANDO EXPRESS.JS*

---

Relatore:  
Chiar.mo Prof. Concetto Spampinato

---

Anno Accademico 2023-2024



# INDICE

|                                                                              |    |
|------------------------------------------------------------------------------|----|
| INTRODUZIONE .....                                                           | 1  |
| 1. WEB PROGRAMMING .....                                                     | 3  |
| 1.1. Architettura delle applicazioni web .....                               | 3  |
| 1.2. Framework per il Web Development.....                                   | 5  |
| 1.3. Panoramica Framework Web esistenti .....                                | 6  |
| 2. EXPRESS.JS E NODE.JS .....                                                | 8  |
| 2.1. Node.js .....                                                           | 8  |
| 2.2. Express.js: Framework per Node.js .....                                 | 11 |
| 2.3. Gestione delle dipendenze con NPM.....                                  | 15 |
| 3. SVILUPPO DEL PANNELLO AMMINISTRATIVO .....                                | 17 |
| 3.1. Struttura dell'applicazione e dipendenze utilizzate .....               | 17 |
| 3.2. Gestione database MySQL e definizione dei modelli con Sequelize ....    | 20 |
| 3.3. Dashboard: autenticazione, middleware e struttura dell'interfaccia..... | 24 |
| 3.4. Product page .....                                                      | 30 |
| 3.5. Order Page .....                                                        | 35 |
| 3.6. Gestione di Utenti, Admin e Messaggi .....                              | 38 |
| CONCLUSIONI .....                                                            | 40 |
| INDICE DELLE FIGURE .....                                                    | 42 |
| SITOGRAFIA.....                                                              | 43 |

# INTRODUZIONE

Da quando è comparso internet nel mondo la programmazione web ha subito un'evoluzione nel corso degli ultimi decenni fino ad oggi con un impatto significativo sulle routine quotidiane delle persone. Il web ha portato grandi cambiamenti ad importantissimi settori come il commercio e l'informazione tra gli altri; ciò ha condotto ad un crescente bisogno di applicazioni web sempre più dinamiche e coinvolgenti. Si può dunque dire che la programmazione per il web svolge un ruolo fondamentale nello sviluppo tecnologico e rappresenta un valido supporto per numerose aziende nell'affrontare le esigenze attuali.

La programmazione web fa riferimento alla combinazione di metodi e linguaggi usati per sviluppare e gestire siti web e applicazioni online. I requisiti degli utenti diventano sempre più complessi, intensificando la richiesta di interfacce utente avanzate, col risultato che sono stati creati framework e librerie varie per semplificare lo sviluppo e aumentare la produttività degli sviluppatori.

Nel processo di sviluppo web è essenziale considerare che un'applicazione debba essere efficiente e facilmente gestibile nel tempo per garantirne la scalabilità a lungo termine. Pertanto, è importante selezionare attentamente il framework più adatto alle esigenze specifiche del progetto e del team di sviluppo. Tra le diverse opzioni disponibili sul mercato emergono Node.js ed Express.js come scelte di grande rilievo per la creazione di applicazioni web lato server grazie alla loro leggerezza e flessibilità nel gestire operazioni asincrone efficacemente.

Node.js è un'innovativa piattaforma di esecuzione di JavaScript che si basa sul motore V8 di Google e ha introdotto un nuovo approccio nello sviluppo lato server consentendo l'uso dello stesso linguaggio sia per la parte front-end che per quella back-end.

Questo ha semplificato il processo di sviluppo e ha ridotto la necessità di conoscere più linguaggi di programmazione, facilitando la vita degli sviluppatori. Node.js si distingue per il suo modello non bloccante, basato su eventi, che consente di gestire tante connessioni simultanee con elevata efficienza.

Express.js, un framework minimalista e flessibile per Node.js, è progettato per facilitare la creazione di applicazioni web e API.

Il progetto descritto in questa tesi si concentra sullo sviluppo di un sito amministrativo, utilizzando Node.js ed Express.js come tecnologie principali. L'obiettivo del progetto è stato quello di creare un'applicazione web amministrativa efficiente, capace di gestire operazioni gestionali più o meno complesse come la gestione degli utenti, la visualizzazione e modifica dei dati con interazioni dinamiche con un database relazionale agenti su un sito di e-commerce già esistente, sviluppato in precedenza. Si è cercato di fornire una panoramica completa, ad un amministratore, di tutte le informazioni cruciali dell'attività come gli utenti registrati, i prodotti presenti sul sito con la possibilità di aggiungere o togliere articoli, la somma delle vendite effettuate, il riepilogo degli ordini effettuati dagli utenti e i messaggi inviati da questi ultimi tramite il sito di e-commerce. Le informazioni poi sono state raccolte e concentrate in una dashboard riassuntiva e interattiva.

La tesi si articola in diversi capitoli, ciascuno dei quali affronta un aspetto specifico del progetto. Nel primo capitolo, viene fornita una panoramica generale del web programming, con un focus particolare sui framework più utilizzati e una comparazione tra le diverse soluzioni disponibili. Il secondo capitolo è dedicato all'analisi approfondita di Node.js ed Express.js, esplorando le loro caratteristiche e i vantaggi che offrono nello sviluppo di applicazioni web moderne. Il terzo capitolo si concentra su come è stato sviluppato il sito amministrativo, descrivendo in dettaglio il codice implementato per ciascuna pagina e le scelte progettuali adottate. Infine, le conclusioni riepilogano i risultati ottenuti, discutono le sfide incontrate durante lo sviluppo e propongono possibili estensioni future del progetto.

# 1. WEB PROGRAMMING

Il web programming, o sviluppo web, è un insieme di metodologie volte allo sviluppo di un'applicazione più o meno complessa. Nel web si possono trovare pagine che contengono contenuti HTML statici o piattaforme con sistemi molto più complessi in cui si ricorre a interagire con risorse esterne come un database o API, come ad esempio i social-network, gli e-commerce o siti di gestione aziendale. Quest'ultimi hanno reso necessario, specialmente negli ultimi anni, di stabilire delle strutture ben precise e delle best practice che consentissero agli sviluppatori di fronteggiare al meglio dell'efficienza le richieste della collettività.

## 1.1. Architettura delle applicazioni web

L'architettura delle applicazioni web gioca un ruolo importante in un sito web. Se si sceglie un'architettura specifica per lo sviluppo della propria app web, si otterranno sicuramente dei vantaggi sia per la manutenzione che per la crescita dell'applicazione. Tuttavia, bisogna fare la scelta giusta nel selezionare l'architettura che fa per la propria app.

La scelta deve ricadere su un'architettura in base a determinati criteri:

- Facile adattamento all'esigenze aziendali: esse possono mutare col tempo e di conseguenza la nostra app deve essere abbastanza flessibile per mantenere il passo;
- Sviluppo organizzato: l'architettura dovrebbe fornire una modularità sufficiente per poter isolare i componenti in base alle necessità, così da avere più elasticità per la scelta della giusta struttura per ogni modulo e componente.
- La sicurezza: la maggior parte delle architetture delle applicazioni web si occupano della sicurezza dei componenti. Si può pianificare in anticipo le misure e le pratiche da implementare per migliorare la sicurezza dell'applicazione prima che venga distribuita.

Prima di approfondire le varie tipologie di architetture adottabili per lo sviluppo di un'applicazione web, è bene esprimere tre concetti chiave del web programming: front-end, back-end e database.

Il front-end è la parte visibile dell'applicazione, con cui gli utenti possono interagire direttamente. Viene sviluppato con tecnologie come HTML, CSS e JavaScript e molto spesso vengono aggiunti dei framework e librerie come Vue.js o React per migliorare la dinamicità e interattività dell'applicazioni web.

Il back-end, invece, è il cuore pulsante di un'applicazione. È qui che si elaborano le richieste provenienti dal front-end, si gestiscono le logiche di business e si interagisce con il database. Con la scelta di un ambiente di sviluppo, come ad esempio Node.js, il back-end può rispondere alle richieste in modo rapido ed efficiente, garantendo la fluidità nell'esperienza dell'utente. In fine, nel database si gestiscono i dati. Può essere relazionale o non relazionale, a seconda delle scelte progettuali scelte. I database relazionali, come MySQL, organizzano i dati in tabelle correlate tra loro attraverso le relazioni. Mentre quelli non relazionali, come MongoDB, offrono un'organizzazione più flessibile e scalabile.

Adesso che si son chiariti quelli che sono gli elementi cardine della programmazione web, è bene elencare alcune delle tipologie di architettura web più diffuse al momento:

1. Client – Server: Un classico, in cui il server risponde alle richieste del client. Usato nella maggior parte dei siti tradizionali.
2. Tree-Tier: Divide il sistema in tre livelli: presentazione(front-end), logica applicativa(back-end) e database.
3. MVC (Model-View-Controller): Struttura che separa l'interfaccia utente (View), i dati (Model) e la logica applicativa (Controller). Ideale per organizzare il codice al meglio.
4. RESTful: Architettura basata su risorse e metodi HTTP (GET, POST, ecc.), molto utilizzata nelle API per la sua semplicità e flessibilità.
5. Serverless: consente di eseguire il codice senza preoccuparsi della gestione di un server. Diventata popolare grazie ai servizi cloud come AWS Lambda.
6. Microservizi: Architettura che divide un'applicazione in servizi indipendenti. Perfetta per progetti grandi e team di sviluppo numerosi.

## 1.2. Framework per il Web Development

Nel panorama attuale del web development, l'uso dei framework è diventato quasi indispensabile per accelerare e semplificare il processo di sviluppo.

Un framework può essere definito come un insieme di strumenti, librerie e best practice predefinite che forniscono una struttura standard per lo sviluppo di applicazioni software. In altre parole, è un supporto che gli sviluppatori utilizzano per costruire applicazioni in modo più efficiente, evitando di dover risolvere da zero problemi comuni. Il framework stabilisce le basi dell'applicazione, permettendo di concentrarsi sugli aspetti specifici del progetto, piuttosto che reinventare funzionalità di base come la gestione delle richieste HTTP, il routing, o l'interazione con il database.

Forniscono anche linee guida che aiutano a mantenere una coerenza stilistica e strutturale nel progetto, facilitando il lavoro di team di sviluppo più grandi.

Nel contesto del web development, si distingue tra framework frontend e framework backend, ognuno con obiettivi e caratteristiche specifiche.

I framework frontend si occupano della parte visibile dell'applicazione, ossia l'interfaccia utente. Sono progettati per facilitare la costruzione di interfacce ricche e dinamiche, che interagiscono con gli utenti in modo fluido e reattivo. Framework come React, Angular, e Vue.js permettono di creare componenti riutilizzabili, migliorando l'efficienza dello sviluppo e l'esperienza utente. Questi framework spesso includono funzionalità per la gestione dello stato dell'applicazione, la navigazione tra le pagine, e l'aggiornamento dinamico del contenuto senza ricaricare la pagina intera.

Dall'altro lato, i framework backend si concentrano sul "dietro le quinte" dell'applicazione. Gestiscono la logica di business, l'autenticazione, la gestione delle sessioni, l'interazione con il database, e l'elaborazione delle richieste provenienti dal frontend. Framework come Express.js per Node.js, Django per Python, e Laravel per PHP forniscono una serie di strumenti che facilitano la creazione di server robusti e scalabili. Essi astraggono molte delle complessità legate alla gestione dei protocolli HTTP, alla sicurezza, e alla connessione con i



database, permettendo agli sviluppatori di concentrarsi sulle funzionalità specifiche del loro progetto.

La scelta di un framework, sia per il frontend che per il backend, dipende dalle esigenze specifiche del progetto, dalle preferenze del team di sviluppo, e dalla comunità di supporto disponibile. In ogni caso, l'adozione di un framework rappresenta un passo fondamentale per garantire un processo di sviluppo più rapido, ordinato, e sostenibile nel lungo termine.

### 1.3. Panoramica Framework Web esistenti

Il community nel web offre una vasta gamma di framework. Questi strumenti si sono evoluti nel tempo per adattarsi alle necessità sempre crescenti degli sviluppatori, sia per quanto riguarda la parte backend che frontend delle applicazioni.

Sul lato backend, alcuni dei framework più noti includono:

**Django:** Sviluppato in Python, Django è noto per la sua filosofia "batteries-included", ovvero offre tutto ciò di cui si ha bisogno per sviluppare un'applicazione web completa. È particolarmente apprezzato per il suo potente ORM (Object-Relational Mapping), il sistema di autenticazione integrato e le funzionalità di amministrazione automatizzate. Django promuove lo sviluppo rapido con un forte focus sulla sicurezza e la scalabilità.

**Ruby on Rails:** Basato sul linguaggio Ruby, Rails ha rivoluzionato il web development introducendo il concetto di "Convention over Configuration". Questo approccio riduce la necessità di scrivere configurazioni dettagliate, affidandosi a convenzioni predefinite che velocizzano lo sviluppo. Ruby on Rails è conosciuto per facilitare la creazione di applicazioni web grazie a un ecosistema ricco di gemme (librerie) che ampliano le sue funzionalità.

**Spring:** Un framework per il linguaggio Java, Spring è particolarmente utilizzato per la creazione di applicazioni aziendali complesse. Offre un'infrastruttura robusta per gestire la configurazione, la sicurezza, e l'integrazione con altre tecnologie.

Spring si distingue per la sua flessibilità, permettendo agli sviluppatori di scegliere i componenti di cui hanno bisogno senza dover utilizzare l'intero framework.

Sul fronte frontend, i framework hanno rivoluzionato il modo in cui le interfacce utente vengono sviluppate, rendendo possibile la creazione di esperienze utente dinamiche e reattive. Tra i più popolari troviamo:

**React:** Creato da Facebook, React è una libreria per la costruzione di interfacce utente basata su componenti. Utilizza un approccio dichiarativo che facilita la gestione dello stato dell'applicazione e consente aggiornamenti efficienti grazie al Virtual DOM. React ha una vasta comunità e un ecosistema che include librerie come Redux per la gestione dello stato globale.

**Angular:** Sviluppato da Google, Angular è un framework completo che fornisce una serie di strumenti per costruire applicazioni single-page (SPA). Utilizza TypeScript come linguaggio di sviluppo e offre funzionalità come il data binding bidirezionale, l'iniezione di dipendenze e un sistema modulare che rende l'applicazione scalabile.

**Vue.js:** Vue.js è un framework progressivo che può essere adottato in modo incrementale. È leggero e facile da integrare, pur offrendo potenti funzionalità per lo sviluppo di applicazioni SPA. Vue.js combina il meglio di React e Angular, offrendo una sintassi chiara e un'ottima documentazione che lo rendono ideale sia per progetti semplici che complessi.

Questi framework, sia per il backend che per il frontend, hanno ciascuno i loro punti di forza e debolezza. La scelta del framework giusto dipende dalle esigenze specifiche del progetto, dalle competenze del gruppo di sviluppo e dal tipo di applicazione da realizzare. Tuttavia, ciò che accomuna tutti questi strumenti è la loro capacità di semplificare il processo di sviluppo, migliorando la produttività e la qualità del codice finale.

## 2. EXPRESS.JS E NODE.JS

Nel panorama dello sviluppo web, la gestione del backend richiede strumenti che siano al tempo stesso leggeri, flessibili e facili da integrare con altre tecnologie. In questo contesto, sono nati diversi framework pensati per semplificare la creazione di applicazioni web, riducendo la complessità della gestione delle richieste e permettendo agli sviluppatori di concentrarsi sulla logica dell'applicazione. Uno di questi è proprio Express.js le cui particolarità le affronteremo in questo capitolo.

### 2.1. Node.js

Node, formalmente Node.js, è un ambiente runtime open-source e multiplatforma che consente ai programmatori di sviluppare tutti i tipi di strumenti e applicazioni lato server in JavaScript. L'esecuzione è prevista al di fuori di un browser (ad esempio, in esecuzione direttamente su un computer o un sistema operativo server). Questo approccio elimina le API JavaScript specifiche di un browser aggiungendo, però, supporto per API del sistema operativo più comuni, tra cui HTTP e librerie di file system.

Vediamo gli aspetti chiave di Node.js:

- JavaScript Runtime: Node.js funziona sul motore JavaScript V8, che è anche il motore principale di Google Chrome.
- Single Thread: un'applicazione Node.js opera all'interno di un singolo processo, evitando così il bisogno di inizializzare un nuovo thread per ogni richiesta.
- I/O asincrono: Node.js fornisce un gruppo di primitive I/O asincrone di default. Esse impediscono al codice JavaScript di bloccarsi, assumendo così un comportamento non bloccante.

Grazie alle proprietà, sopra elencate, esso trova applicazioni in vari ambiti tra cui: Server Web, API e microservizi, applicazioni in tempo reale, applicazioni a singola pagina e servizi di streaming.

I motivi per scegliere Node.js come tecnologia back-end, ricade, innanzitutto, sulla sua facilità di utilizzo specialmente per i principianti che si addentrano nella programmazione web. Inoltre, è scalabile sia orizzontalmente che verticalmente, eccellendo anche nella sincronizzazione in tempo reale. Un altro vantaggio non da poco è quello che permette l'utilizzo di un solo linguaggio, JavaScript, sia server-side che client-side. Vantando, infine, di una vasta frontiera di librerie open source. Ora vediamo un esempio pratico di come implementare un semplice server web utilizzando il pacchetto Node HTTP.

Una volta installato Node nella nostra macchina, basta creare una directory con dentro il nostro file in formato js. Utilizzando un qualsiasi editor di testo, possiamo scrivere le seguenti serie di istruzioni:

```
// Load HTTP module
const http = require("http");

const hostname = "127.0.0.1";
const port = 8000;

// Create HTTP server
const server = http.createServer(function (req, res) {
  // Set the response HTTP header with HTTP status and Content type
  res.writeHead(200, { "Content-Type": "text/plain" });

  // Send the response body "Hello World"
  res.end("Hello World\n");
});

// Prints a log once the server starts listening
server.listen(port, hostname, function () {
  console.log(`Server running at http://${hostname}:${port}/`);
});
```

**Figura 2.1 - Creazione server web con Node.js**

Il server sarà in ascolto sulla porta e sul nome host specificati. Quando arriverà una nuova richiesta, la funzione di callback la gestisce impostando lo stato della risposta, le intestazioni e il contenuto. In questo caso, accedendo al

**http://localhost:8000** in un browser web, vedremo il testo “Hello World” in alto a sinistra della nostra finestra.

In generale, il funzionamento consiste nel fatto che Node.js accetta la richiesta dai client e invia la risposta, tale richiesta viene gestita con un singolo thread.

Il thread è una sequenza di istruzioni che il server deve eseguire. Grazie al fatto che Node.js è un linguaggio single-thread event loop, può gestire richieste simultanee con un singolo thread senza bloccarsi per una richiesta.

L’event loop, dunque, è la caratteristica che rende Node così performante. Il suo funzionamento è semplice. In sostanza, esso monitora una serie di code, che prendono il nome di queue, in cui vengono inseriti i compiti da eseguire, ad esempio il completamento di richieste al server, la lettura di file o altre attività asincrone. L’event loop controlla queste code ciclicamente e processa i task quando sono pronti, permettendo così al programma di continuare a funzionare senza interruzioni.

Nonostante i numerosi vantaggi, Node.js presenta alcune limitazioni che è bene approfondire, soprattutto quando si tratta di scegliere la tecnologia giusta per un progetto. Il modello single-thread, che costituisce uno dei suoi principali punti di forza, può rivelarsi inadatto in contesti in cui le operazioni da eseguire si spostano dall’I/O asincrono all’uso intensivo della CPU per via di elaborazioni di immagini, calcoli matematici complessi o algoritmi di crittografia. In tali situazioni, il thread principale rischia di bloccarsi, causando grossi rallentamenti all’intero sistema.

Per affrontare questa problematica, Node.js mette a disposizione il modulo Worker Threads, che consente di eseguire codice JavaScript su thread separati. I Worker Threads rappresentano una trovata efficace per le operazioni che gravano sulla CPU, distribuendo il carico di lavoro su più thread e garantendo così una maggiore efficienza.

Un’altra possibile limitazione di Node.js è data dal fatto che, basandosi su un motore runtime veloce come V8, il suo funzionamento ottimale dipende dall’hardware sottostante. Sebbene questo sia raramente un problema per la maggior parte dei sistemi moderni, in casi particolari potrebbe essere necessario ottimizzare l’hardware per garantire delle adeguate performance.

Nonostante questi ostacoli, le soluzioni offerte da Node.js, come i Worker Threads e il supporto della comunità, ne confermano la versatilità e l'efficacia nel contesto del web development.

## **2.2. Express.js: Framework per Node.js**

Express.js è un framework di backend Node.js minimalista e offre funzionalità e strumenti solidi per lo sviluppo di applicazioni di backend scalabili. Express fornisce una serie di strumenti per applicazioni web, le richieste e le risposte HTTP, il routing e il middleware per la creazione e la distribuzione di applicazioni su larga scala e di livello aziendale.

Fornisce inoltre uno strumento di interfaccia a riga di comando (CLI) chiamato Node Package Manager (NPM), da cui si possono reperire i pacchetti sviluppati dalla comunità. Inoltre, obbliga gli sviluppatori a seguire il principio di Don't Repeat Yourself (DRY). Il principio DRY punta a ridurre le ridondanze dei pattern del software, sostituendoli con astrazioni o con la normalizzazione dei dati per evitare ripetizioni.

Inoltre, grazie all'astrazione da parte di Express di gran parte di questi complessi sistemi, permette agli sviluppatori di concentrarsi maggiormente sulla creazione delle funzionalità pensate per la propria applicazione. Da ciò deriva il fatto che è meglio affidarsi a Express.js che utilizzare il solo Node.js nel backend.

Express.js viene utilizzato per un'ampia gamma di scopi: con Express è possibile sviluppare applicazioni, endpoint API, sistemi di routing e framework. Di seguito elenchiamo solo alcuni dei possibili sistemi che si possono creare con Express.js.

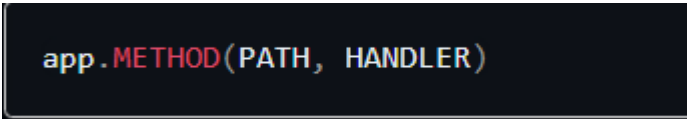
- Applicazioni a Pagina Singola: le applicazioni SPA costituiscono un nuovo approccio allo sviluppo di applicazioni in cui l'intera applicazione viene indirizzata in un'unica pagina. Express.js è ben indicato per creare un'API da connettere a queste applicazioni SPA e fornire i dati in modo coerente. Alcuni esempi famosi sono Gmail, Google Maps, Netflix e molti altri. Le aziende utilizzano le SPA per dare agli utenti un'esperienza fluida e scalabile.

- Strumenti di Collaborazione in Tempo Reale: gli strumenti di collaborazione semplificano il lavoro interno alle aziende permettendo una facile ed efficiente collaborazione tra le varie parti interne e con Express.js è possibile sviluppare facilmente applicazioni rivolte al networking in tempo reale come applicazioni di chat e dashboard.
- Applicazioni di Streaming: le applicazioni di Streaming in tempo reale come Netflix sono complesse e presentano molti livelli di flussi dati. Per creare una app di questo tipo è necessario un framework solido che gestisca al meglio i flussi dati asincroni. Express si presta abbastanza bene per adempiere a questo scopo.
- Applicazioni Fintech: il termine Fintech indica un programma informatico e altri strumenti utilizzate a supporto di servizi bancari e finanziari. Express.js è il framework preferito per lo sviluppo di applicazioni di questo tipo.

Oltre a ereditare da Node.js il sistema di single thread e l'asincronia tra i processi, Express.js utilizza un'architettura MVC per semplificare la manipolazione dei dati e i sistemi di routing. Il routing consiste nel determinare il modo in cui un'applicazione risponde a una richiesta client rivolta a un endpoint specifico con uno specifico metodo di richiesta HTTP (GET, POST, e così via).

Ogni percorso può avere una o più funzioni di gestione, che vengono eseguite quando l'utente richiede un determinato percorso.

La definizione di una rotta assume la seguente struttura:



```
app.METHOD(PATH, HANDLER)
```

Figura 2.2 - Definizione rotta su Express.js

Dove: **app** è un'istanza di express, **METHOD** è un metodo di richiesta HTTP, **PATH** è il percorso della risorsa nel server e **HANDLER** è la funzione eseguita quando il percorso viene trovato.

Il sistema di routing di Express.js è utile per gestire la struttura dell'applicazione, raggruppando le varie rotte in un'unica cartella.

Il framework, inoltre, comprende una serie di middleware che consentono di creare delle funzioni fluide e che non interrompono il normale flusso dell'applicazione.

I middleware sono delle funzioni che vengono eseguite prima che una richiesta HTTP raggiunga il controller associato al percorso richiesto, o diversamente, prima che il client riceva una risposta. La funzione ha accesso a tre parametri tra cui: l'oggetto richiesta (req), l'oggetto risposta (res) e ad una variabile che punta al successivo middleware da eseguire, comunemente denominata "next". I middleware possono svolgere svariati compiti tra cui: eseguire qualsiasi codice, modificare gli oggetti richiesta e risposta, terminare il ciclo richiesta-risposta e chiamare la funzione successiva nello stack.

Vediamo un esempio di definizione di un middleware configurato in modo da essere eseguito ad ogni richiesta:

```
const express = require('express')
const app = express()

app.use((req, res, next) => {
  console.log('Time:', Date.now())
  next()
})
```

**Figura 2.3 - Definizione middleware in Express.js**

Con i middleware, gli sviluppatori possono intercettare, tramite degli script il flusso dell'applicazione; ad esempio, si possono usare i middleware per verificare se un utente, prima di accedere a un determinato percorso, sia autorizzato o meno.



Creiamo un semplice server che ascolti le richieste in arrivo da uno specifico URL e numero di porta con Express.js:

```
const express = require('express')
const app = express()
const port = 4000

app.get('/', (request, response) => {
  response.send('Testing Hello World!')
})

app.listen(port, () => {
  console.log(`Test app listening at http://localhost:${port}`)
})
```

Figura 2.4 - Creazione di un server con Express.js

Questo semplice server ascolterà le richieste provenienti su **http://localhost:4000/** e restituirà una risposta testuale “Testing Hello World!”.

In conclusione, Express.js si conferma uno strumento molto valido per lo sviluppo di applicazioni web scalabili e robuste. Grazie alla sua struttura minimalista e alla possibilità di utilizzare dei middleware personalizzabili, rende possibile la creazione facilitata di applicazioni altamente efficienti. L’inglobare Node.js permette di sfruttare appieno i vantaggi portati da quest’ultimo, mentre l’architettura MVC semplifica la gestione dei dati e delle rotte. Grazie al supporto ai vari ambiti citati in questo paragrafo, questo framework si consolida come uno dei framework più utilizzati per lo sviluppo backend. La sua flessibilità e la comunità di supporto lo rendono un framework adatto sia per i principianti che per gli sviluppatori più esperti.

## 2.3. Gestione delle dipendenze con NPM

NPM (Node Package Manager) è un gestore di pacchetti sviluppato appositamente per la piattaforma JavaScript Node.js. La sua funzione principale è quella di consentire agli sviluppatori open-source di condividere il proprio codice attraverso il registro npm. Durante la progettazione di un'applicazione, spesso è necessario installare pacchetti che migliorano e ampliano le funzionalità della nostra applicazione web. Una volta che i moduli vengono installati vengono registrati in un file fondamentale chiamato `package.json`.

Il file `package.json` è anche chiamato il manifesto del progetto. Esso tiene traccia di tutti i pacchetti installati, fornendo informazioni come nome, versione, autore e così via. Inoltre, definisce attributi importanti utilizzati da npm per installare dipendenze ed eseguire script.

La generazione del file `package.json` avviene tramite il comando: **npm init**.

Una volta creato il file, è possibile iniziare a installare i pacchetti necessari tramite il comando: **npm install <nome\_pacchetto>**.

Ad esempio, per installare il framework Express, basta eseguire: `npm install express`.

Dopo l'installazione, `package.json` includerà una nuova sezione chiamata "dependencies", che elenca tutte le librerie essenziali, che noi abbiamo installato, necessarie per il corretto funzionamento del nostro progetto.

Le dipendenze in Node.js si suddividono in due macrocategorie:

1. Dipendenze di produzione: le dipendenze di produzione sono fondamentali per il funzionamento del progetto in ambiente reale. Vengono elencate nel `package.json` nella sezione `dependencies`. Questi pacchetti sono direttamente utilizzati nel codice sorgente dell'applicazione e vengono installati automaticamente se non presenti nella cartella contenenti i moduli di Node.js installati.
2. Dipendenze di sviluppo: le dipendenze di sviluppo, invece, sono pacchetti utili esclusivamente durante la fase di sviluppo ma non necessari per la produzione. Vengono registrate nel file di `package.json` nella sezione `devDependencies` e non vengono incluse nel codice finale distribuito agli

utenti. Un esempio è rappresentato dal pacchetto Nodemon che riavvia automaticamente l'applicazione Node.js quando rileva le modifiche. Per aggiungere una dipendenza di sviluppo si utilizza il comando:

**npm install <nome\_pacchetto> --save-dev.**

Vediamo adesso un esempio di file package.json:

```
{
  "name": "adminpanel-express",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "start": "nodemon app.js"
  },
  "keywords": [],
  "author": "Giuseppe Ravesi",
  "license": "ISC",
  "description": "Progetto Tesi - Ingegneria Informatica L-8",
  "devDependencies": {
    "nodemon": "^3.1.7"
  },
  "dependencies": {
    "bcrypt": "^5.1.1",
    "connect-flash": "^0.1.1",
    "ejs": "^3.1.10",
    "express": "^4.21.0",
    "express-fileupload": "^1.5.1",
    "express-session": "^1.18.0",
    "mysql2": "^3.11.3",
    "sequelize": "^6.37.4"
  }
}
```

Figura 2.5 - package.json del Progetto

In questo esempio si possono notare le varie sezioni, prima citate, riguardo le dipendenze come anche le altre informazioni fondamentali del progetto, tra cui il nome dell'app, autore e così via. L'esempio fa riferimento all'effettivo manifesto del progetto trattato in questa tesi, le quali dipendenze verranno descritte nel successivo capitolo.

## 3. SVILUPPO DEL PANNELLO AMMINISTRATIVO

In questo capitolo vedremo la descrizione dettagliata del progetto inerente alla creazione di un sito che vada a supportare la gestione di un e-commerce di vendita ristorativa. Il sito offre una vasta gamma di funzionalità volte al monitoraggio delle vendite e gestione dei prodotti presenti sul sito. Il pannello di amministrazione è pensato come strumento ad uso esclusivo del proprietario. Dopo il primo accesso, che avviene tramite credenziali fittizie, l'utente ha la possibilità di modificarle. Inoltre, è stata prevista la possibilità per il super admin di creare ulteriori account amministrativi, da assegnare eventualmente ai propri dipendenti. Dal punto di vista progettuale si è adottata un'architettura MVC, in cui le responsabilità sono ben suddivise tra modello, vista e controller, facilitando così la gestione e la manutenzione del codice.

Per l'implementazione è stato scelto Express.js come framework back-end, sia per i vantaggi illustrati nei capitoli precedenti, sia perché la sua struttura è simile a quella di Laravel, framework trattato nel corso di programmazione web e utilizzato per lo sviluppo dell'e-commerce che il nostro AdminPanel andrà a supportare.

Per la gestione dei dati, il progetto si appoggia ad un database relazionale, MySQL, oltre a fare uso di vari middleware per la gestione dell'autenticazione degli utenti e moduli specifici per la creazione di modelli relazionali, gestione delle sessioni su express, il rendering delle viste e così via. Nelle prossime sezioni analizzeremo la struttura dell'applicazione, le dipendenze (moduli) utilizzati e la descrizione delle singole pagine del sito amministrativo.

### 3.1. Struttura dell'applicazione e dipendenze utilizzate

Dopo la creazione e l'inizializzazione del progetto in Express.js, la struttura di base risulta piuttosto essenziale. Il progetto inizialmente include il file **package.json**, il file di blocco delle dipendenze **package-lock.json**, la cartella **node\_modules** e il

file principale **app.js**, all'interno del quale viene configurato il server. Un esempio di configurazione di base è stato mostrato in *Figura 2.4*.

Basandoci sull'architettura MVC adottata per il progetto, sono state create tre cartelle principali: **controllers**, **models** e **views**, rispettivamente dedicate alla gestione della logica dell'applicazione, dei modelli di dati e del rendering delle pagine. Oltre a queste, sono state aggiunte due ulteriori cartelle:

- **public**, che contiene tutte le risorse lato client, come file CSS per lo stile, script JavaScript e immagini da mostrare sul sito;
- **routes**, in cui risiede il file **web.js**, nel quale sono definite tutte le rotte previste per l'applicazione.

All'interno di **controllers**, è stata aggiunta un'altra directory, **middlewares**, che contiene la definizione di due script necessari per l'ottimale gestione di autenticazioni e mantenimento di dati cruciali degli utenti.

Questa organizzazione ha permesso di mantenere il codice ben strutturato e facilmente gestibile, garantendo una chiara separazione delle responsabilità tra i vari componenti previsti.

Per estendere la funzionalità offerte da Express.js si è visto necessario utilizzare diverse dipendenze, installate tramite npm.

Tra le principali dipendenze utilizzate troviamo:

- Express.js: il framework principale utilizzato per gestire il routing e il middleware dell'applicazione.
- Express-session: modulo per la gestione delle sessioni utente.
- EJS (Embedded JavaScript): motore di template per il rendering dinamico delle pagine.
- MySQL2: dipendenza per la connessione al database MySQL.
- Sequelize: strumento ORM Node.js per supportare le interazioni con il database relazionale.
- Express-fileupload: semplice middleware per il caricamento di file.

Tali dipendenze vengono importate e configurate nel file **app.js**. Di seguito, un estratto di codice sorgente che mostra l’inizializzazione dei principali moduli:

```
const express = require('express');
const session = require('express-session');
const app = express();
const fileUpload = require('express-fileupload');
app.use(fileUpload());
app.use(express.static('public'));

app.set('view engine', 'ejs');

app.use(session({
  secret: '39dk3ld93fkls02m',
  resave: false,
  saveUninitialized: false,
  cookie: { maxAge: 3600000 }
}));
//route
app.use('/', require('./routes/web'));

app.listen(3000);
```

Figura 3.1 - App.js, inizializzazione server web e moduli

In *Figura 3.1* il codice esposto mostra come Express venga inizializzato, come le sessioni siano gestite tramite express-session, e come EJS venga impostato come motore di rendering delle viste.

Inoltre, con **app.use(express.static('public'))**, si stanno rendendo accessibili i file statici contenuti nella cartella **public**, servendoli automaticamente senza dover definire una rotta specifica per ciascuno.

Con **app.use('/', require('./routes/web'))**; definiamo il percorso in cui trovare le rotte che Express userà per gestire tutte le richieste che arrivano alla root del sito (ovvero il percorso '/'). In fine, il server viene messo in ascolto nella porta 3000.

Si è scelto di centralizzare la configurazione del database e del modulo Sequelize in un file dedicato nominato **db.js** di cui parleremo in seguito.

La struttura così impostata ha permesso di rendere il progetto modulabile e ordinato così da facilitare la gestione e manutenzione delle singole componenti dell'applicazione.

## 3.2. Gestione database MySQL e definizione dei modelli con Sequelize

L'AdminPanel è un'applicazione web di supporto al sito di e-commerce, il quale aveva una base dati relazionale MySQL. Dunque, si è scelto di continuare a utilizzare il database MySQL già esistente aggiungendo però una tabella che contenesse le informazioni relative agli account degli admin. Non si è visto necessario creare una nuova base dati per il momento.

Per semplificare le interazioni con il database si è scelto di utilizzare un ORM (Object Relational Mapper) basato su Node.js, Sequelize. Un ORM esegue funzioni come la gestione dei record del database rappresentando i dati come oggetti. Sequelize ha un potente meccanismo di migrazione, fornisce un eccellente supporto per la sincronizzazione del database, il caricamento rapido dei dati, riducendo di gran lunga i tempi di sviluppo e prevenendo le iniezioni SQL.

Per poter integrare Sequelize bisogna innanzitutto aver installato e configurato MySQL. Una volta fatto è possibile creare una connessione al database attraverso un'istanza di Sequelize, nel seguente modo:

```
const { Sequelize } = require('sequelize');

const sequelize = new Sequelize('hw2_db', 'root', '', {
  host: 'localhost',
  dialect: 'mysql',
});

sequelize.authenticate()
  .then(() => {
    console.log('Connessione al database riuscita!');
  })
  .catch(err => {
    console.error('Errore nella connessione al database:', err);
    process.exit(1);
  });

module.exports = sequelize;
```

Figura 3.2 - Configurazione database con Sequelize

In *Figura 3.2*, vediamo come creare un'istanza di Sequelize con **new Sequelize()**, in cui si passano, come parametri, il nome del database, il nome utente del server MySQL e le credenziali del database. Con l'attributo **host** si definisce dove è

ospitato il server MySQL, che nel nostro caso è installato localmente. Il metodo **authenticate()** è utilizzato per connettersi al database e verifica se le credenziali fornite sono corrette. La connessione al database, all'avvio del server, rimarrà sempre aperta e non sarà necessario ristabilirla per ogni query che si intende eseguire. La maggior parte dei metodi forniti da Sequelize sono asincroni. Ciò vale a dire che le operazioni con il database risulteranno non bloccanti. Sequelize ritornerà una **promise**, se tutto è andata a buon fine, pertanto, si possono utilizzare i metodi **then()**, **catch()**, e **finally()** per restituire i dati elaborati.

Abbiamo visto come si instaura una connessione con il database relazionale, adesso, parleremo della struttura dei modelli Sequelize nel nostro progetto. Un modello è un'astrazione che rappresenta una tabella del database. Per definire un modello, bisogna identificare il nome della tabella, i dettagli delle colonne e i tipi di dati presenti.

Proceduralmente, la creazione di un model avviene in due step:

1. Viene importato il modello Sequelize, creato in **db.js**, così da instaurare la connessione con il database. Inoltre, è necessario integrare l'oggetto **DataTypes** che fornisce un vasto assortimento di tipo di dato da poter assegnare ai campi dei nostri modelli:

```
const { DataTypes } = require('sequelize');
const sequelize = require('../db');
```

Figura 3.3

2. Si procede alla creazione del modello attraverso **sequelize.define()** in cui si definiranno le varie colonne della tabella, definendo per ciascuna varie proprietà previste in un database relazionale (tipo, primary key e ecc.).



Vediamo, come esempio, la definizione di uno dei model del progetto:

```
const Admin = sequelize.define('Admin', {
  id: {
    type: DataTypes.INTEGER,
    allowNull: false,
    primaryKey: true,
    autoIncrement: true,
  },
  name: {
    type: DataTypes.STRING(20),
    allowNull: false,
  },
  password: {
    type: DataTypes.STRING(50),
    allowNull: false,
  },
  super:{
    type: DataTypes.BOOLEAN,
    allowNull: false,
  }
}, {
  tableName: 'admin',
  timestamps: false,
});
```

Figura 3.4 - Definizione modello Admin

L'esempio vede la definizione del modello che fa riferimento alla tabella admin nel database. Per ogni campo vediamo necessariamente la definizione del tipo che estrapoliamo dall'oggetto `DataTypes` precedentemente importato. Nel codice, **allowNull** permette di dichiarare se un campo in un record può essere NULL.

Il progetto prevede la definizione dei seguenti modelli, corrispondenti alle tabelle del database relazionale:

- **Admin:** rappresenta ogni amministratore registrato nel pannello di gestione.
- **Cart:** funge da tabella di supporto per associare gli utenti ai prodotti che intendono acquistare.
- **Message:** memorizza i messaggi inviati dagli utenti tramite la piattaforma.
- **Order:** traccia tutti gli ordini effettuati dagli utenti.
- **Product:** identifica ciascun prodotto disponibile per la vendita.
- **User:** rappresenta gli utenti registrati sul sito di e-commerce.

Per avere un quadro più chiaro della struttura del database, si fornisce il diagramma entità-relazioni:

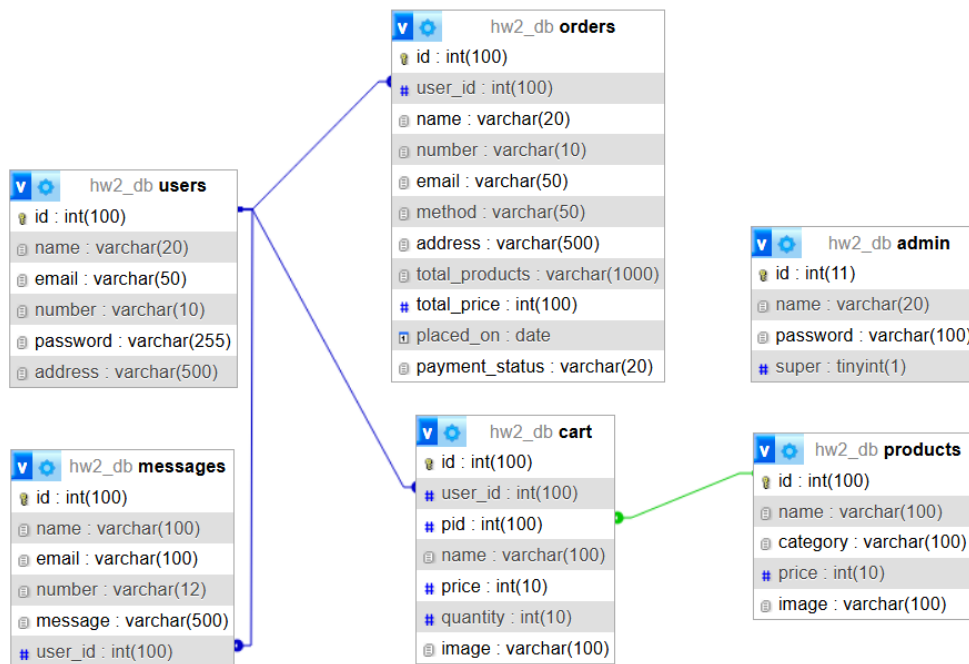


Figura 3.5 - Diagramma entità/relazione del db

La struttura del database in figura è piuttosto semplice. La tabella **users** è in relazione uno-a-molti con **cart**, così come lo è **products**. Inoltre, **users** è collegata a **messages**, sebbene non sia obbligatorio essere registrati per inviare un messaggio; tuttavia, quando l'utente è registrato, si è scelto di associare il suo ID al messaggio. Infine, la tabella **orders** presenta una relazione molti-a-uno con **users**.

In Sequelize sarebbe stato possibile definire queste relazioni direttamente nei modelli, ma nel contesto del progetto non si è reso necessario, quindi si è preferito non implementarle.

Con questo abbiamo ultimato la descrizione strutturale del progetto. Nei prossimi paragrafi si andrà ad approfondire la realizzazione delle varie pagine del sito gestionale, analizzando anche l'impiego dei modelli trattati in questo paragrafo.

### 3.3. Dashboard: autenticazione, middleware e struttura dell'interfaccia

La Dashboard rappresenta la pagina principale dell'applicazione. In essa sono visibili, in sezioni ben distinte, tutte le informazioni riguardanti il sito di e-commerce. Inoltre, a partire da essa, è possibile navigare in tutte le pagine presenti nell'applicazione attraverso una navbar inserita nell'header della pagina. Quest'ultimo, è una vista trattata come componente, dato che è ricorrente in tutte le pagine del sito, si è scelto di scrivere il codice in un file esterno, da richiamare quando serve.

Per poter accedere alla rotta della dashboard bisogna prima autenticarsi. La gestione dell'accesso alle pagine amministrative viene implementata attraverso un middleware di autenticazione chiamato **checkAuthentication**.

La logica seguita dal middleware è la seguente:

1. Esclusione delle pagine di login
  - Se la richiesta è per **/admin\_login** o **/login\_admin**, il middleware permette l'accesso senza eseguire ulteriori controlli, chiamando direttamente **next()**.
2. Verifica della sessione
  - Se l'utente è autenticato (**req.session.isLoggedIn** è **true**), viene effettuato un ulteriore controllo per verificare i permessi di accesso a determinate pagine.
  - Se l'utente cerca di accedere alle rotte **/admin\_accounts** o **/user\_account**, ma non è un super admin, viene mostrato un messaggio di errore e l'utente viene reindirizzato alla dashboard.
3. Accesso negato per utenti non autenticati
  - Se l'utente non è loggato, viene visualizzato un messaggio e viene reindirizzato alla pagina di login.

Questo middleware è essenziale per proteggere il pannello di gestione e distinguere tra admin e super admin. L'implementazione viene esportata e richiamata nel file del server, applicandola a tutte le rotte, tranne quelle di autenticazione che sono le uniche pagine a cui si può accedere se non autenticati.

```
const checkAuthentication = (req, res, next) => {
  if (req.path === '/admin_login' || req.path === '/login_admin') {
    return next();
  }
  if (req.session.isLoggedIn) {
    if (req.path === '/admin_accounts' || req.path === '/user_account') {
      if (!req.session.admin.super) {
        req.flash('message', 'Devi essere super admin per accedere a questa pagina');
        res.redirect('/dashboard');
      }
    }
    next();
  } else {
    req.flash('message', 'Devi loggarti per accedere a questa pagina');
    res.redirect('/admin_login');
  }
}

module.exports = checkAuthentication;
```

Figura 3.6 - middleware di autenticazione

Le variabili di sessioni, trattate in *Figura 3.6*, sono settate nel momento in cui l'utente effettua il login. Nella sessione vengono memorizzati:

- `isLoggedIn`: una variabile booleana che serve a verificare l'effettiva autenticazione
- `admin`: un oggetto contenente tre attributi come l'id dell'admin, il nome utente e una variabile booleana **super** che serve a distinguere tra admin e super admin.

Poiché i dati dell'admin devono essere a disposizione per tutta la durata della sessione, questi vengono salvati all'interno dell'oggetto **req.session.admin**. Tuttavia, questa soluzione ha un limite: le variabili di sessione sono accessibili solo lato server, mentre per utilizzarle nel rendering delle pagine lato client è necessario un passaggio intermedio. Per questo scopo, è stato implementato un ulteriore middleware **getAdminProfile**, il cui compito è quello di rendere accessibili i dati degli admin direttamente nelle viste. Il middleware verifica se **req.session.admin** è definito e, in caso affermativo, copia il suo contenuto nell'oggetto

**res.locals.admin.** L'oggetto in questione è una variabile globale fornita da Express e resa disponibile automaticamente al template di rendering (EJS). Dopo eseguita questa operazione, il middleware chiama **next()**, proseguendo così la normale esecuzione della richiesta. Questa soluzione ci permette di accedere facilmente ai dati admin all'interno dei template senza doverla passare manualmente a ogni rotta che effettua una richiesta.

Introdotte queste funzionalità, possiamo parlare dell'effettiva implementazione dell'header come componente esterno e la dashboard.

La dashboard permette di ottenere una panoramica generale dei dati essenziali come il numero di ordini in sospeso e completati, il totale degli utenti registrati, il numero di prodotti disponibili e i messaggi ricevuti.

Per la sua realizzazione, sono stati utilizzati Sequelize per il recupero dei dati dal database e EJS per la renderizzazione dinamica della pagina.

L'accesso alla dashboard è gestito dalle rotte definite in **web.js**:

```
const loadDashboardData = [  
  dashboardController.getDashboardData,  
  dashboardController.renderDashboard  
];  
  
router.get('/', loadDashboardData);  
router.get('/dashboard', loadDashboardData);
```

**Figura 3.7**

Queste due rotte garantiscono che la dashboard venga fornita sia quando si accede alla root: **'/'** sia tramite l'URL: **/dashboard**.

Le informazioni visibili sulla dashboard vengono aggiornate dal **DashboardController**, che esegue due funzioni principali:

1. Recupero dei dati dal database (**getDashboardData**).
2. Renderizzazione della pagina (**renderDashboard**).

Il metodo **getDashboardData** pesca le informazioni necessarie interrogando il database con Sequelize:

```
const getDashboardData = async (req, res, next) => {
  try {
    const [totalPendings, totalCompleted, totalOrders, totalProducts, users,
    admins, messages] = await Promise.all([
      Order.findAll({ where: { payment_status: 'pending' } }),
      Order.findAll({ where: { payment_status: 'COMPLETED' } }),
      Order.findAll(),
      Product.findAll(),
      User.findAll(),
      Admin.findAll(),
      Message.findAll()
    ]);

    req.total_pendings = totalPendings.reduce((sum, order) => sum + order.to-
    tal_price, 0);
    req.total_completed = totalCompleted.reduce((sum, order) => sum + order.to-
    tal_price, 0);
    req.totalOrders = totalOrders;
    req.totalProducts = totalProducts;
    req.users = users;
    req.admins = admins;
    req.messages = messages;
    next();
  } catch (error) {
    console.error('Errore durante il recupero dei dati della dashboard:', er-
    ror);
    res.status(500).send('Errore durante il recupero dei dati della da-
    shboard.');
```

**Figura 3.8 - Estrapolazione dei dati da visualizzare nella Dashboard**

Questa funzione esegue più query in parallelo grazie a **Promise.all()**, migliorando l'efficienza della richiesta. I dati estrapolati vengono poi memorizzati in **req** per essere accessibili nella fase successiva.

Successivamente, **renderDashboard** si occupa di inviare i dati alla vista EJS:

```
const renderDashboard = (req, res) => {
  res.render('dashboard', {
    total_pendings: req.total_pendings || '0',
    total_completed: req.total_completed || '0',
    total_orders: req.totalOrders.length || '0',
    numbers_of_products: req.totalProducts.length || '0',
    number_users: req.users.length || '0',
    number_admins: req.admins.length || '0',
    number_messages: req.messages.length || '0'
  });
};
```

**Figura 3.9 - Renderizzazione dei dati nella Dashboard**

Questa funzione passa le informazioni alla pagina **dashboard.ejs**, che le utilizza per visualizzare i dati in modo dinamico.

Il file **dashboard.ejs** è il template che definisce la struttura della pagina amministrativa. Qui vengono visualizzati i dati recuperati dal backend, utilizzando la sintassi di EJS per trattare i dati.

Ad esempio, questa sezione mostra il numero totale di ordini completati e in sospeso:

```
<div class="box">
  <h3><span>${total_pendings}</span></h3>
  <p>ordini in coda</p>
  <a href="/placed_orders?status=pending" class="btn">vedi ordini</a>
</div>

<div class="box">
  <h3><span>${total_completed}</span></h3>
  <p>ordini completati</p>
  <a href="/placed_orders?status=COMPLETED" class="btn">vedi ordini</a>
</div>
```

**Figura 3.10 - Esempio disposizione elementi nel template EJS**

Questa struttura utilizza i dati passati dal controller (**total\_pendings** e **total\_completed**) per aggiornare in tempo reale il contenuto della dashboard.

Dato che l'header è presente in più pagine del pannello amministrativo, è stato separato in file EJS riutilizzabile: **admin\_header.ejs**.

Questa scelta ha permesso di includere dinamicamente l'header in più pagine con una semplice riga nel front-end:

```
<%- include('./admin_header') %>
```

Figura 3.11 - Sintassi EJS per includere componenti esterni in una vista

L'header fornisce i collegamenti alle pagine dell'AdminPanel e mostra le informazioni dell'admin loggato.

```
<section class="flex">

  <a href="/dashboard" class="logo">Admin<span>Panel</span></a>

  <nav class="navbar">
    <a href="/dashboard">dashboard</a>
    <a href="/products">prodotti</a>
    <a href="/placed_orders">ordini</a>
    <a href="/admin_accounts">admin</a>
    <a href="/user_account">utenti</a>
    <a href="/messages">messaggi</a>
  </nav>

  <div class="icons">
    <div id="menu-btn" class="fas fa-bars"></div>
    <div id="user-btn" class="fas fa-user"></div>
  </div>

  <div class="profile">
    <p><%= admin.name %></p>
    <a href="/update_profile" class="btn">modifica profilo</a>
    <div class="flex-btn">
      <!--<a href="/admin_login" class="option-btn">login</a-->
      <a href="/admin_register" class="option-btn">registrati</a>
    </div>
    <a href="/admin_logout" onclick="return confirm('Vuoi davvero uscire?');" class="logout-btn">logout</a>
  </div>

</section>
```

Figura 3.12 - Header del progetto

Inoltre, l'header accoglie tutti i messaggi di notifica dai vari controller così che gli utenti possono visualizzarli e interagire con loro per chiuderli. La gestione di questo meccanismo avviene tramite il controllo di una variabile globale (message) e se è settata, il contenuto viene visualizzato all'interno di un div a comparsa associato a un'icona che al click permette la scomparsa del contenitore. Questo comportamento viene gestito da uno script inline.



L'implementazione della dashboard sfrutta una chiara separazione tra backend e frontend, con il controller che gestisce la logica lato server e il template EJS che si occupa della rappresentazione della pagina. L'uso di un header riutilizzabile migliora la modularità del codice, rispettando il principio DRY imposto da Express.js.

### 3.4. Product page

La Product Page rappresenta il punto chiave del sito, in cui vengono visualizzati i prodotti disponibili sulla piattaforma e-commerce, oltre a permettere operazioni CRUD (creazione, aggiornamento ed eliminazione) sui prodotti. La parte backend è gestita dal **ProductController** e definito nelle rotte, mentre, utilizziamo un template EJS per il rendering dinamico dei dati nel frontend.

Nel file delle rotte (web.js) sono definite le rotte interessate alla visualizzazione e modifica dei prodotti, ovvero:

```
router.get('/products', productController.getProducts);
router.post('/addProducts', productController.addProduct);
router.delete('/products/:id', productController.deleteProduct);
router.get('/update_product/:id', productController.getProducts);
router.post('/updateProduct', productController.updateProduct);
```

Figura 3.13 - Definizione rotte Product Page

Queste rotte sono gestite dal **ProductController**, che si occupa di:

- Recuperare i prodotti dal database e passarli alla vista, tramite **getProducts**.
- Aggiungere nuovi prodotti usando il metodo **addProduct**, che raccoglie i dati del form, gestisce il caricamento delle immagini e interagisce con il database.
- Aggiornare o eliminare prodotti tramite le relative funzioni, che eseguono operazioni CRUD utilizzando il modulo Sequelize.

Vediamo in particolare come sono state implementate le funzioni **addProduct** e **deleteProduct**.

La funzione **addProduct** riceve i dati inviati dal form (**req.body** e **req.files**), li valida e, se tutto è corretto, salva il prodotto nel database, gestendo anche l'upload dell'immagine corrispondente nel server locale.

Inizialmente serve estrarre i dati dal form:

```
const { name, price, category } = req.body;
```

Figura 3.14 - Esempio di destructuring assignment

Qui si utilizza la destructuring assignment di JavaScript per estrarre i valori **name**, **price** e **category** da **req.body**, che contiene i dati inviati dal client tramite una richiesta POST.

Successivamente, i valori ricevuti vengono validati con il metodo **.trim()** e si verifica se il prodotto che si intende inserire nella piattaforma non esiste già:

```
const sanitizedName = name.trim();
const sanitizedPrice = price.trim();
const sanitizedCategory = category.trim();

const existingProduct = await Product.findOne({ where: { name: sanitizedName } });
if (existingProduct) {
  req.flash('message', 'Nome prodotto già esistente');
  return res.redirect('/products');
}
```

Figura 3.15 - Validazione e verifica dei dati su addProduct

Nel codice usiamo il metodo **findOne()** di Sequelize, che cerca un record nel database che rispetti una condizione specifica (in questo caso, un prodotto con lo stesso nome). Se il prodotto esiste già, viene visualizzato un messaggio di errore e l'utente viene reindirizzato alla pagina dei prodotti.

L'utente nella compilazione del form per aggiungere il prodotto può caricare un'immagine. Nel codice, la funzione controlla se è stato effettivamente inviato un file prima di procedere al suo salvataggio, ciò avviene come segue:

```
if (req.files && req.files.image) {
  const image = req.files.image;
  if (image.size > 2000000) {
    req.flash('message', 'La dimensione dell\'immagine è troppo grande');
    return res.redirect('/products');
  }
}
```

Figura 3.16 - Trattazione dell'immagine del prodotto

L'oggetto **req.files**, in *Figura 3.16*, contiene tutti i file caricati dal client, ed è messo a disposizione dal modulo **express-fileupload**. Questo middleware analizza le richieste HTTP che contengono file (form con **enctype="multipart/form-data"**) e li rende accessibili tramite **req.files**. Ogni chiave dell'oggetto rappresenta un file caricato. Nel nostro caso, se il form ha un campo **<input type="file" name="image">**, il file sarà accessibile con la chiave **req.files.image**.

Inoltre, per evitare errori, si effettuano dei controlli in testa per assicurarsi che l'oggetto e la chiave siano stati effettivamente definiti.

Se le verifiche sono tutte positive si effettua un'istanza del record nel database:

```
try {
  await Product.create({
    name: sanitizedName,
    category: sanitizedCategory,
    price: sanitizedPrice,
    image: image.name
  });

  req.flash('message', 'Nuovo prodotto aggiunto con successo!');
  res.redirect('/products');
} catch (dbError) {
  console.error('Errore durante inserimento del prodotto:', dbError);
  req.flash('message', 'Errore durante l'inserimento del prodotto, riprova più tardi.');
```

**Figura 3.17** - Inserimento dati con Sequelize

La funzione **deleteProduct** si pone come obiettivo quello di rimuovere un prodotto dal database e se presenti, eliminare anche i riferimenti ad esso in altre tabelle (come il carrello). In questa funzione si è scelto che l'invio dei dati per l'eliminazione di un prodotto, non avvenga tramite form HTML inviato al server, ma tramite una richiesta AJAX con **fetch()** da Javascript.

Lato server la funzione gestisce le richieste DELETE sulla rotta **/products/:id**.

Vediamo i passaggi chiave della logica di implementazione:

- Recupero dell'ID del prodotto dalla richiesta:

```
const deleteId = req.params.id;
if (!deleteId) {
  return res.status(400).json({ message: 'ID prodotto non fornito' });
}
const product = await Product.findByPk(deleteId);
if (!product) {
  return res.status(404).json({ message: 'Prodotto non trovato' });
}
```

**Figura 3.18**

- Verifica dell'esistenza del prodotto nel database
- Eliminazione del prodotto e delle sue associazioni:

```
await Product.destroy({ where: { id: deleteId } });
await Cart.destroy({ where: { pid: deleteId } });

res.status(200).json({ message: 'Prodotto eliminato con successo' });
```

il prodotto viene eliminato dalla tabella Product, e se esiste un riferimento in Cart, anche questi record vengono cancellati. Se tutto va a buon fine rispondiamo con un oggetto JSON, a differenza di un classico **res.redirect()**.

La pagina **products.ejs** mostra l'elenco dei prodotti attualmente presenti nel database e offre un'interfaccia per l'aggiunta di nuovi articoli.

Nel codice si ha la netta separazione delle due sezioni, che per quanto riguarda il form:

```
<form action="/addProducts" method="POST" enctype="multipart/form-data">
  <h3>Aggiungi Prodotto</h3>
  <input type="text" required placeholder="inserisci nome prodotto" name="name" maxLength="100" class="box">
  <input type="number" min="0" max="9999999999" required placeholder="inserisci prezzo prodotto" name="price"
    | onkeypress="if(this.value.length == 10) return false;" class="box">
  <select name="category" class="box" required>
    <option value="" disabled selected>selezione categoria --</option>
    <option value="main dish">primo</option>
    <option value="fast food">fast food</option>
    <option value="drinks">bevanda</option>
    <option value="desserts">dolce</option>
  </select>
  <input type="file" name="image" class="box" accept="image/jpg, image/jpeg, image/png, image/webp" required>
  <input type="submit" value="aggiungi prodotto" name="add_product" class="btn">
</form>
```

**Figura 3.19 - form inserimento prodotto**

Mentre per quanto riguarda l'elenco dei prodotti:

```
<div class="box-container">
  <% if (products && products.length > 0) { %>
    <% products.forEach(function(fetch_products) { %>
      <div class="box" id="product-<%= fetch_products.id %>">
        
        <div class="flex">
          <div class="price"><span><%= fetch_products.price %></span></div>
          <div class="category"><%= fetch_products.category %></div>
        </div>
        <div class="name"><%= fetch_products.name %></div>
        <div class="flex-btn">
          <a href="/update_product/<%= fetch_products.id %>" class="option-btn">Modifica</a>
          <button class="delete-btn" data-id="<%= fetch_products.id %>">Elimina</button>
        </div>
      </div>
    <% }); %>
  <% } else { %>
    <p class="empty">Non ci sono prodotti nel sito</p>
  <% } %>
</div>
```

**Figura 3.20 - disposizione box dei prodotti**

Nella *Figura 3.20*, in particolare, viene mostrato come, utilizzando la sintassi EJS, vengono generati dinamicamente i box prodotti all'interno di un flex-container. Il server restituisce un elenco di prodotti, e con un ciclo li si scorre uno per uno, creando, per ciascuno, un div che rappresenta un prodotto. Ad ogni box, oltre alle informazioni come nome e prezzo, viene associato un pulsante "Elimina". Esso ha un data attribute che contiene l'ID del prodotto corrispondente. Nel file JavaScript associato alla pagina, responsabile di inviare al server quale prodotto eliminare, fa in modo di intercettare il click su questi pulsanti, recuperando l'ID del prodotto specifico. Una volta ottenuto l'ID, lo script invia una richiesta AJAX con il metodo DELETE evitando così un refresh della pagina.

Grazie a questo approccio, l'eliminazione dei prodotti avviene in modo fluido e dinamico, migliorando l'esperienza utente.

### 3.5. Order Page

In questa pagina l'amministratore è in grado di visualizzare tutti gli ordini effettuati nella piattaforma. Gli ordini che arrivano possono essere conclusi, cioè che è stato effettuato il pagamento online, o in attesa di essere conclusi poiché il cliente ha scelto di effettuare un pagamento alla consegna. In questo ultimo caso, sarà successivamente l'admin a concludere l'ordine dalla pagina degli ordini. Nel database relazionale, tra i vari campi, teniamo traccia dello stato dell'ordine nel campo "payment\_status" e distinguiamo i due casi con "completed" in caso di avvenuto pagamento o "pending" in caso contrario.

La pagina degli ordini è implementata seguendo lo schema logico delle altre pagine.

Nel backend, la gestione degli ordini avviene attraverso tre operazioni principali:

- Recupero degli ordini dal database, con la possibilità di filtrare per stato di pagamento.
- Aggiornamento dello stato di pagamento per ordini esistenti.
- Eliminazione di un ordine dal database, se non è stato ancora completato.

Il **recupero degli ordini** avviene tramite la funzione **getOrders()** nel controller **OrderController**, che una volta pescati i dati li rende insieme alla vista.

Il codice è il seguente:

```
const getOrders = async (req, res) => {
  const status = req.query.status;
  let orders;
  try {
    if (status) {
      orders = await Order.findAll({ where: { payment_status: status } });
    } else {
      orders = await Order.findAll();
    }
    return res.render('placed_orders', { orders, status });
  } catch (error) {
    console.error('Errore durante il recupero degli ordini:', error);
    res.status(500).send('Errore durante il recupero degli ordini.');
  }
}
```

Figura 3.21 – getOrder

Se il client passa un parametro status nella query string, vengono recuperati tutti gli ordini che rispecchiano quella condizione. Se così non fosse, verrebbero recuperati

tutti gli ordini. Il client può filtrare gli ordini attraverso la dashboard in cui sono presenti tre sezioni. In base a quale box l'utente clicca, la variabile status, passata attraverso la query string, assume valori diversi: **completed** o **pending** o non assumerne affatto e quindi ottenere tutti gli ordini presenti nella tabella orders.

**L'aggiornamento dello stato di pagamento** da parte del client avviene attraverso una richiesta POST al server, che viene risolta attraverso il metodo **updatePaymentStatus()**. Vediamo l'implementazione:

```
const updatePaymentStatus = async (req, res) => {
  try {
    const { order_id, payment_status } = req.body;
    if (!order_id || !payment_status) {
      req.flash('message', 'ID ordine o stato di pagamento non forniti');
      return res.redirect('/placed_orders');
    }
    await Order.update(
      { payment_status },
      { where: { id: order_id } }
    );
    req.flash('message', 'Stato di pagamento aggiornato con successo!');
    res.redirect('/placed_orders');
  } catch (error) {
    console.error('Errore durante l\'aggiornamento dello stato di pagamento:',
error);
    req.flash('message', 'Si è verificato un errore durante l\'aggiornamento
dello stato di pagamento, riprova più tardi.');
```

Figura 3.22 - Metodo di aggiornamento stato ordine

Il funzionamento non è complesso, la funzione riceve dal client l'**order\_id** e il nuovo **payment\_status** e controlla l'effettiva esistenza dei due parametri. Se risulta essere positivo, aggiorna il database usando Sequelize (Order.update).

**L'eliminazione di un ordine**, invece, viene gestito attraverso la funzione **deleteOrder** il quale funzionamento può essere descritto attraverso dei semplici step:

- Legge l'**id** dell'ordine dalla URL (/placed\_orders/delete/1).
- Se l'**id** non è presente, mostra un messaggio di errore.
- Altrimenti, cancella l'ordine dal database usando **Order.destroy()**.
- Infine, renderizza l'utente con un messaggio di conferma.

Lato client, il file **placed\_orders.ejs** è responsabile della visualizzazione degli ordini:

```
<section class="placed-orders">
  <h1 class="heading">Ordini</h1>
  <div class="box-container">
    <% if (orders && orders.length > 0) { %>
      <% orders.forEach(function(order) { %>
        <div class="box">
          <p> user id : <span><%= order.user_id %></span> </p>
          <p> effettuato in data : <span><%= order.placed_on %></span> </p>
          <p> nome : <span><%= order.name %></span> </p>
          <p> email : <span><%= order.email %></span> </p>
          <p> cellulare : <span><%= order.number %></span> </p>
          <p> indirizzo : <span><%= order.address %></span> </p>
          <p> prodotti totali : <span><%= order.total_products %></span> </p>
          <p> totale speso : <span><%= order.total_price %></span> </p>
          <p> metodo di pagamento : <span><%= order.method %></span> </p>

          <% if (order.payment_status === 'COMPLETED') { %>
            <p> stato di pagamento : <span><b>Completato</b></span> </p>
          <% } else { %>
            <form action="/update_payment_status" method="POST">
              <input type="hidden" name="order_id" value="<%= order.id %>">
              <select name="payment_status" class="drop-down">
                <option value="pending">pending</option>
                <option value="COMPLETED">COMPLETED</option>
              </select>
              <div class="flex-btn">
                <input type="submit" value="aggiorna" class="btn"
name="update_payment">
                <a href="/placed_orders?delete=<%= order.id %>"
class="delete-btn">elimina</a>
              </div>
            </form>
          <% } %>
        </div>
      <% }); %>
    <% } else { %>
      <p class="empty">nessun ordine piazzato</p>
    <% } %>
  </div>
</section>
```

**Figura 3.23 - Pagine degli ordini lato client**

Come visto per la pagina dei prodotti, sull'Order Page si itera su orders, creando una scheda per ogni ordine. Se lo stato di pagamento è completato, non viene permessa la visualizzazione di aggiornamento o il pulsante di eliminazione, viceversa, viene mostrato un form con una selezione in cui è possibile modificare lo stato di pagamento insieme a un pulsante di elimina per cancellare l'ordine.



### 3.6. Gestione di Utenti, Admin e Messaggi

Oltre alle sezioni principali, il sito include pagine di gestione per utenti, amministratori e messaggi. In ognuna di queste pagine, il backend si occupa di recuperare i dati dal database e renderizzarli al client. Nel frontend, i dati verranno iterati e distribuiti nelle pagine di appartenenza in blocchi dinamici che si adattano alle dimensioni della viewport e al numero di blocchi di aggiungere. Inoltre, ogni voce ha la possibilità di essere eliminata tramite un'operazione diretta, che viene gestita attraverso una richiesta al server, come abbiamo già visto nelle precedenti pagine.

Le funzionalità previste per ciascuna pagina sono:

- Users: la pagina mostra l'elenco degli utenti registrati con le informazioni essenziali (ID, e-mail, ecc.), permettendo agli admin di eliminarli.
- Admin: elenca gli account amministrativi. Qui, ha l'accesso solo il super admin, dove ha la possibilità di registrare o eliminare gli account degli admin.
- Messages: permette la visualizzazione dei messaggi ricevuti dagli utenti e permette agli admin di eliminarli una volta letti.
- Update Admin Profile: rende possibile ad ogni account admin di aggiornare le informazioni inerenti al proprio profilo come nome utente e password.

L'eliminazione delle voci è gestita in modo asincrono lato client, utilizzando una richiesta AJAX con fetch, senza il bisogno di ricaricare l'intera pagina. Questo migliora l'esperienza utente, rendendo l'interfaccia più fluida. Dopo l'eliminazione, il relativo elemento viene rimosso dal DOM dinamicamente con JavaScript, aggiornando la visualizzazione.

La pagina dedicata alla modifica del profilo admin consente agli amministratori di aggiornare le proprie informazioni. Data la sensibilità delle informazioni gestite, sono state adottate misure di sicurezza per garantire l'aggiornamento dei dati senza rischi o vulnerabilità. Il backend verifica che, nel caso in cui si volesse modificare il nome utente, esso non esista già e che la modifica della password avvenga correttamente, confrontando la vecchia password con quella salvata nel database, applicando l'hashing sulla nuova. L'aggiornamento avviene in modo sincrono con

una semplice richiesta POST, mentre lato client il form impone alcune restrizioni sui campi, come la rimozione degli spazi vuoti e il controllo della lunghezza massima. Come nelle altre pagine, quest'ultime forniscono feedback immediato all'utenti tramite messaggi flash, migliorando l'esperienza di utilizzo.

Vediamo il controller dell'aggiornamento del profilo admin:

```
const updateAdminProfile = async (req, res) => {
  try {
    const adminId = req.session.admin.id;
    const { name, old_pass, new_pass, confirm_pass } = req.body;
    if (name && name.trim()) {
      const sanitized_name = name.trim();
      const existingAdmin = await Admin.findOne({ where: { name: sanitized_name }
});
      if (existingAdmin) {
        req.flash('message', 'Username già utilizzato!');
        return res.redirect('/update_profile');
      } else {
        await Admin.update({ name: sanitized_name }, { where: { id: adminId } });
        req.flash('message', 'Nome aggiornato con successo!');
      }
    }
    if (old_pass && new_pass && confirm_pass) {
      const admin = await Admin.findByPk(adminId);
      if (!admin) {
        req.flash('message', 'Profilo admin non trovato!');
        return res.redirect('/update_profile');
      }
      const isOldPasswordValid = await bcrypt.compare(old_pass, admin.password);
      if (!isOldPasswordValid) {
        req.flash('message', 'Vecchia password non corretta!');
        return res.redirect('/update_profile');
      } else if (new_pass !== confirm_pass) {
        req.flash('message', 'La password di conferma non coincide!');
        return res.redirect('/update_profile');
      } else {
        const hashedPassword = await bcrypt.hash(new_pass, 10);
        await Admin.update({ password: hashedPassword }, { where: { id: adminId }
});
        req.flash('message', 'Password aggiornata con successo!');
      }
    }
    res.redirect('/update_profile');
  } catch (error) {
    console.error("Errore durante l'aggiornamento del profilo admin:", error);
    req.flash('message', 'Si è verificato un errore, riprova più tardi.');
```

**Figura 3.24 - Controller Aggiornamento profilo admin**

## CONCLUSIONI

Durante lo sviluppo del progetto, Express.js si è rivelato essere davvero utile per gestire il backend. La sua semplicità e varietà di middleware hanno permesso di strutturare il progetto in modo modulare ed efficiente. Grazie a Express, è stato possibile definire rapidamente le rotte del server, la gestione delle richieste HTTP e le interazioni con il database senza eccessiva complessità.

Tuttavia, le difficoltà non sono mancate. Una delle principali sfide è stata capire la gestione ottimale del caricamento delle immagini lato server. Nella logica implementata, l'upload delle immagini avviene utilizzando `express-fileupload`. Il codice in sé non si è rivelato assai complicato, la difficoltà sta, inizialmente, nella comprensione della gestione dei percorsi accessibili al server, senza causare conflitti nei nomi dei file e la gestione degli errori durante il trasferimento. Inoltre, il salvataggio locale delle immagini, rappresenta un approccio poco scalabile in un contesto di produzione con un numero eccessivo di file. Una futura soluzione a questa problematica, che per il fine del progetto può andare anche bene, è quella di adottare un servizio cloud storage come AWS S3, che permetterebbe di delegare la gestione dei file a un sistema più affidabile, oltre a ridurre il carico del server.

Un'altra difficoltà è stata riscontrata nella realizzazione di operazioni asincrone con il database, risolte approfondendo il meccanismo di `async/await` e nella corretta gestione degli errori.

Pur essendo completamente funzionante, il sito può essere migliorato sotto diversi aspetti. Uno dei primi riguarda l'interfaccia utente: al momento il design è funzionale e responsive a qualsiasi risoluzione, ma potrebbe essere reso più all'avanguardia utilizzando un framework front-end.

Un'altra possibile ottimizzazione sarebbe la gestione dei messaggi di notifica e di errore, attualmente implementati con il modulo `flash messages`. Sarebbe più efficace integrare un sistema di notifiche in tempo reale utilizzando `WebSockets` per aggiornare dinamicamente lo stato degli ordini e altre informazioni rilevanti.

Il progetto, così com'è stato sviluppato, può essere esteso in diversi modi. Una prima possibile estensione potrebbe essere lo sviluppo di un app mobile che

permetta agli amministratori di gestire il sito attraverso il proprio smartphone. Un'altra possibile direzione potrebbe riguardare l'implementazione di funzionalità di analisi avanzata, come statistica sulle vendite e previsioni sull'acquisto basate su modelli di machine learning. Rendendo così l'app non solo uno strumento di gestione ma anche un supporto strategico di marketing.

## INDICE DELLE FIGURE

|                                                                                |    |
|--------------------------------------------------------------------------------|----|
| Figura 2.1 - Creazione server web con Node.js .....                            | 9  |
| Figura 2.2 - Definizione rotta su Express.js.....                              | 12 |
| Figura 2.3 - Definizione middleware in Express.js .....                        | 13 |
| Figura 2.4 - Creazione di un server con Express.js .....                       | 14 |
| Figura 2.5 - package.json del Progetto .....                                   | 16 |
| Figura 3.1 - App.js, inizializzazione server web e moduli .....                | 19 |
| Figura 3.2 - Configurazione database con Sequelize.....                        | 20 |
| Figura 3.3 .....                                                               | 21 |
| Figura 3.4 - Definizione modello Admin.....                                    | 22 |
| Figura 3.5 - Diagramma entità/relazione del db .....                           | 23 |
| Figura 3.6 - middleware di autenticazione.....                                 | 25 |
| Figura 3.7 .....                                                               | 26 |
| Figura 3.8 - Estrapolazione dei dati da visualizzare nella Dashboard .....     | 27 |
| Figura 3.9 - Renderizzazione dei dati nella Dashboard .....                    | 28 |
| Figura 3.10 - Esempio disposizione elementi nel template EJS .....             | 28 |
| Figura 3.11 - Sintassi EJS per includere componenti esterni in una vista ..... | 29 |
| Figura 3.12 - Header del progetto .....                                        | 29 |
| Figura 3.13 - Definizione rotte Product Page .....                             | 30 |
| Figura 3.14 - Esempio di destructuring assignment .....                        | 31 |
| Figura 3.15 - Validazione e verifica dei dati su addProduct.....               | 31 |
| Figura 3.16 - Trattazione dell'immagine del prodotto .....                     | 31 |
| Figura 3.17 - Inserimento dati con Sequelize .....                             | 32 |
| Figura 3.18 .....                                                              | 32 |
| Figura 3.19 - form inserimento prodotto .....                                  | 33 |
| Figura 3.20 - disposizione box dei prodotti .....                              | 34 |
| Figura 3.21 – getOrder.....                                                    | 35 |
| Figura 3.22 - Metodo di aggiornamento stato ordine .....                       | 36 |
| Figura 3.23 - Pagine degli ordini lato client .....                            | 37 |
| Figura 3.24 - Controller Aggiornamento profilo admin .....                     | 39 |

## SITOGRAFIA

Cos'è l'architettura delle applicazioni web? URL: <https://kinsta.com/it/blog/architettura-applicazioni-web/>

"Web Architecture 101" di Jonathan Fulton URL: <https://medium.com/storyblocks-engineering/web-architecture-101-a3224e126947>

Angular vs. React vs. Vue.js: Comparing performance URL: [https://blog.logrocket.com/angular-vs-react-vs-vue-js-comparing-performance/?utm\\_source=chatgpt.com](https://blog.logrocket.com/angular-vs-react-vs-vue-js-comparing-performance/?utm_source=chatgpt.com)

Comparing the best new JavaScript frameworks to React URL: [https://blog.logrocket.com/comparing-the-best-new-javascript-frameworks-to-react/?utm\\_source=chatgpt.com](https://blog.logrocket.com/comparing-the-best-new-javascript-frameworks-to-react/?utm_source=chatgpt.com)

Documentazione ufficiale di Django URL: <https://docs.djangoproject.com>

Documentazione ufficiale di Spring Framework URL: <https://spring.io/projects/spring-framework>

Introduzione a Node.js, URL: <https://www.geeksforgeeks.org/node-js-introduction/>

Express/Node introduction, URL:

[https://developer.mozilla.org/enUS/docs/Learn\\_web\\_development/Extensions/Server-side/Express\\_Nodejs/Introduction](https://developer.mozilla.org/enUS/docs/Learn_web_development/Extensions/Server-side/Express_Nodejs/Introduction)

Event loop Node.js URL: <https://nodejs.org/en/learn/asynchronous-work/event-loop-timers-and-nexttick>

Fast, unopinionated, minimalist web framework for Node.js URL: <https://expressjs.com/>

Cos'è Express.js? Tutto quello che c'è da sapere URL: <https://kinsta.com/it/knowledgebase/cos-e-express-js/>

Worker Threads in Node.js URL: [https://nodejs.org/api/worker\\_threads.html](https://nodejs.org/api/worker_threads.html)

Basic routing URL: <https://expressjs.com/en/starter/basic-routing.html>

Using middleware URL: <https://expressjs.com/en/guide/using-middleware.html>

Npm Docs URL: <https://docs.npmjs.com/>

Sequelize Docs URL: <https://sequelize.org/docs/v6/getting-started/>

How to use Sequelize with Node.js e MySQL URL:  
<https://www.digitalocean.com/community/tutorials/how-to-use-sequelize-with-node-js-and-mysql>

Destructuring assignment URL: [https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Destructuring\\_assignment](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Destructuring_assignment)